



RECIPICK

YOLO11N과 OPEN AI API를 활용한 식재료 인식 및 레시피 추천

딥러닝 2조

20225202 김가은

20200331 서동민

20202645 손찬우

20220335 조은나라

목차

프로젝트 소개

중간 발표 Review

데이터 구성

모델 선택 및 모델 성능 평가

OpenAI API 기반 레시피 추천 시스템

결론

프로젝트 소개



프로젝트명: recipick

1인 가구의 식재료 관리 불편 개선
냉장고 속 재료를 이미지로 촬영하면
→ YOLOv11n 모델이 재료 인식
→ OpenAI GPT가 실제 가능한 레시피 추천

시스템 전체 흐름

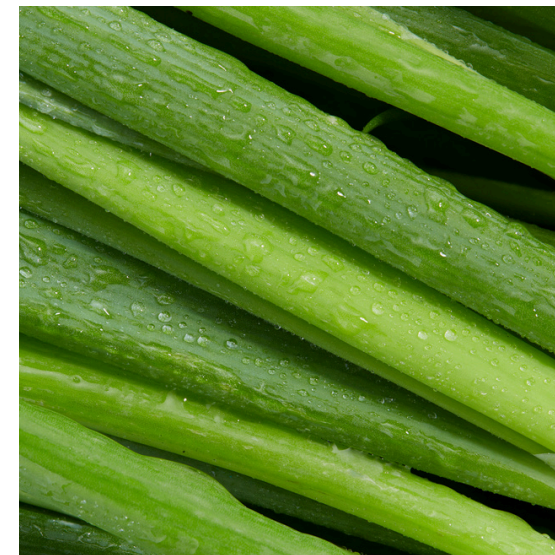
이미지 → YOLOv11n detection
→ GPT가 레시피 추천 → 결과 출력

목표

인식 정확도와 실용적인 추천 기능을 모두 갖춘 레시피 추천 스마트 시스템 구축

문제 정의

- 1인 가구 증가 → 식재료 방치·폐기 증가
- 어떤 요리를 할 수 있는지 스스로 판단 어려움
- 기존 앱들은 직접 재료 입력을 요구함 → 번거로움
- 냉장고 내부 인식 기능은 고가·특수 제품에만 탑재

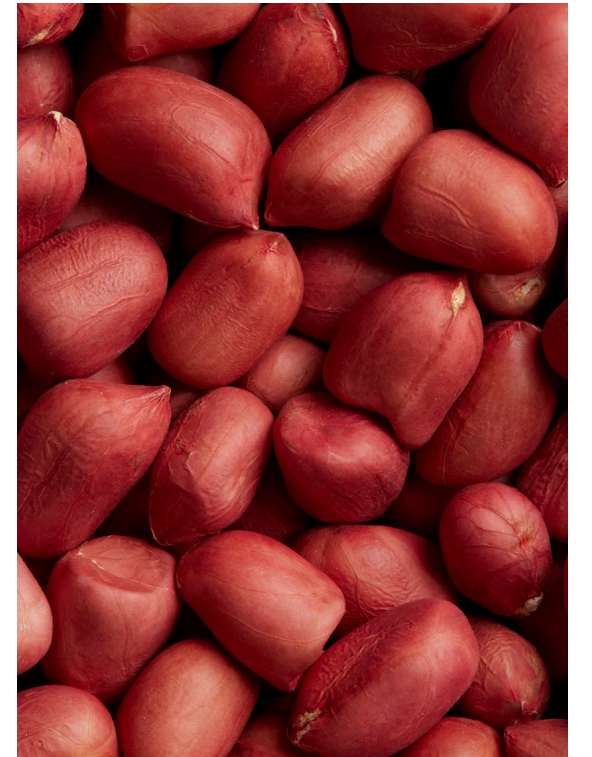


fresh



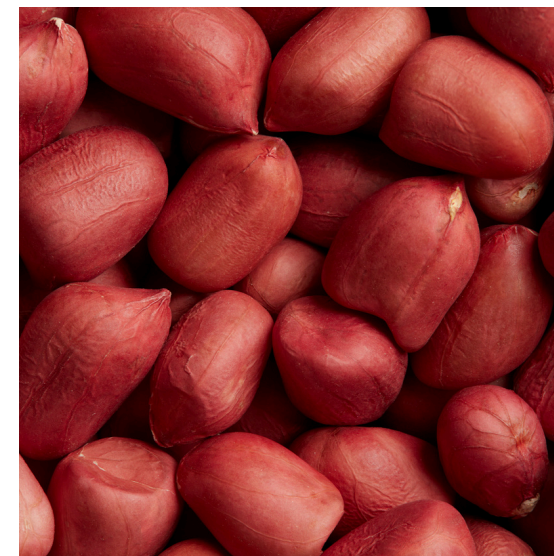
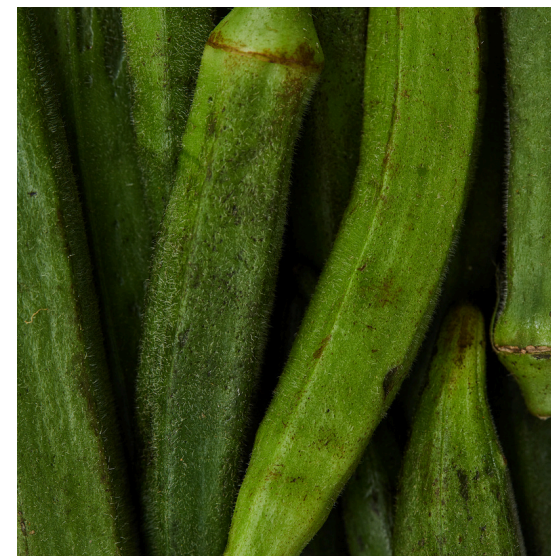
중간 발표 review

| DEEP LEARNING



중간 발표 이후 변경된 점

- 초기 계획: OID 데이터 + YOLOv8 기반 모델
- 중간 발표 후 새롭게 판단:
- → OID 데이터는 1인 가구 식재료와 맞지 않음
- 데이터 전략 전면 변경
- 모델도 YOLOv11로 최신 버전 업그레이드



데이터 전략 변경

데이터 구성 변화

- OID 데이터 → 사용하지 않음
- ChatGPT에게 “1인 가구가 자주 사용하는 식재료 리스트” 생성 요청
- 그 결과를 기반으로 최종 클래스 20여 종 선정
- 즉, 데이터셋이 목적에 맞게 완전 맞춤형으로 변환됨



데이터 수집 과정

데이터 수집

- 팀원 4명이 식재료 목록 나눠 맡아 직접 수집
- Roboflow에서 이미지 확보 + 라벨링된 데이터 다운로드
- train/val/test = 7 : 2 : 1 비율 유지
- 이후 Python 병합 코드로 전체 데이터셋 통합

```
!pip install roboflow
```

```
from roboflow import Roboflow
rf = Roboflow(api_key="RnkjmnscvCLTJKJcYLUT")
project = rf.workspace("curry").project("sprout")
version = project.version(1)
dataset = version.download("yolov11")
```

```
# =====
# 2. 클래스 통합 및 ID 매핑
# =====
print("=== 1. 클래스 ID 통합 분석 ===")

yaml_my_path = os.path.join(path_my, "data.yaml")
if os.path.exists(yaml_my_path):
    with open(yaml_my_path, 'r', encoding='utf-8') as f:
        classes_my = yaml.safe_load(f)['names']
else:
    classes_my = []
    print("⚠ 내 데이터셋 yaml 파일 없음 (빈 리스트로 시작)")

merged_classes = list(classes_my)
for cls in classes_team:
    if cls not in merged_classes:
        merged_classes.append(cls)

print(f"🌟 최종 클래스 개수: {len(merged_classes)}개")

map_my = {old: merged_classes.index(name) for old, na
map_team = {old: merged_classes.index(name) for old,

print("✅ ID 매핑 준비 완료")
```

```
# =====
# 3. 유틸리티 함수 (해시 & 경로 길이 체크)
# =====
def get_file_hash(file_path):
    """파일의 내용을 읽어서 MD5 해시값을 반환"""
    hash_md5 = hashlib.md5()
    with open(file_path, "rb") as f:
        for chunk in iter(lambda: f.read(4096), b''):
            hash_md5.update(chunk)
    return hash_md5.hexdigest()

processed_hashes = set()
duplicate_count = 0
renamed_count = 0

# =====
# 4. 데이터 병합 (KeyError 해결됨)
# =====
print("\n=== 2. 병합 및 중복 제거 시작 ===")

splits = ['train', 'val', 'test']

# ★ 핵심 수정: 'valid' 키를 사용하지 않고 'val'로 통일하되, 실제 폴더는 둘 다 있음 ★
folder_candidates = {
    'train': ['train'],
    'val': ['val', 'valid'],
    'test': ['test']
}

if os.path.exists(output_path):
    print(f"🔪 기존 폴더 정리 중... ({output_path})")
    shutil.rmtree(output_path)

abs_output_path = os.path.abspath(output_path)

for sp in splits:
    os.makedirs(f"{output_path}/{sp}/images", exist_ok=True)
    os.makedirs(f"{output_path}/{sp}/labels", exist_ok=True)

def process_dataset(root_path, source_name, id_map, structure_type):
    global duplicate_count, renamed_count
    print(f"\n📁 [{source_name}] 처리 중...")

    for split in splits:
        target_src_dir = None
        src_lbl_base = None

        if structure_type == 1: # 내 데이터 (train/images)
            # 내 데이터는 'val'이 없으면 'valid' 폴더를 찾으려
```

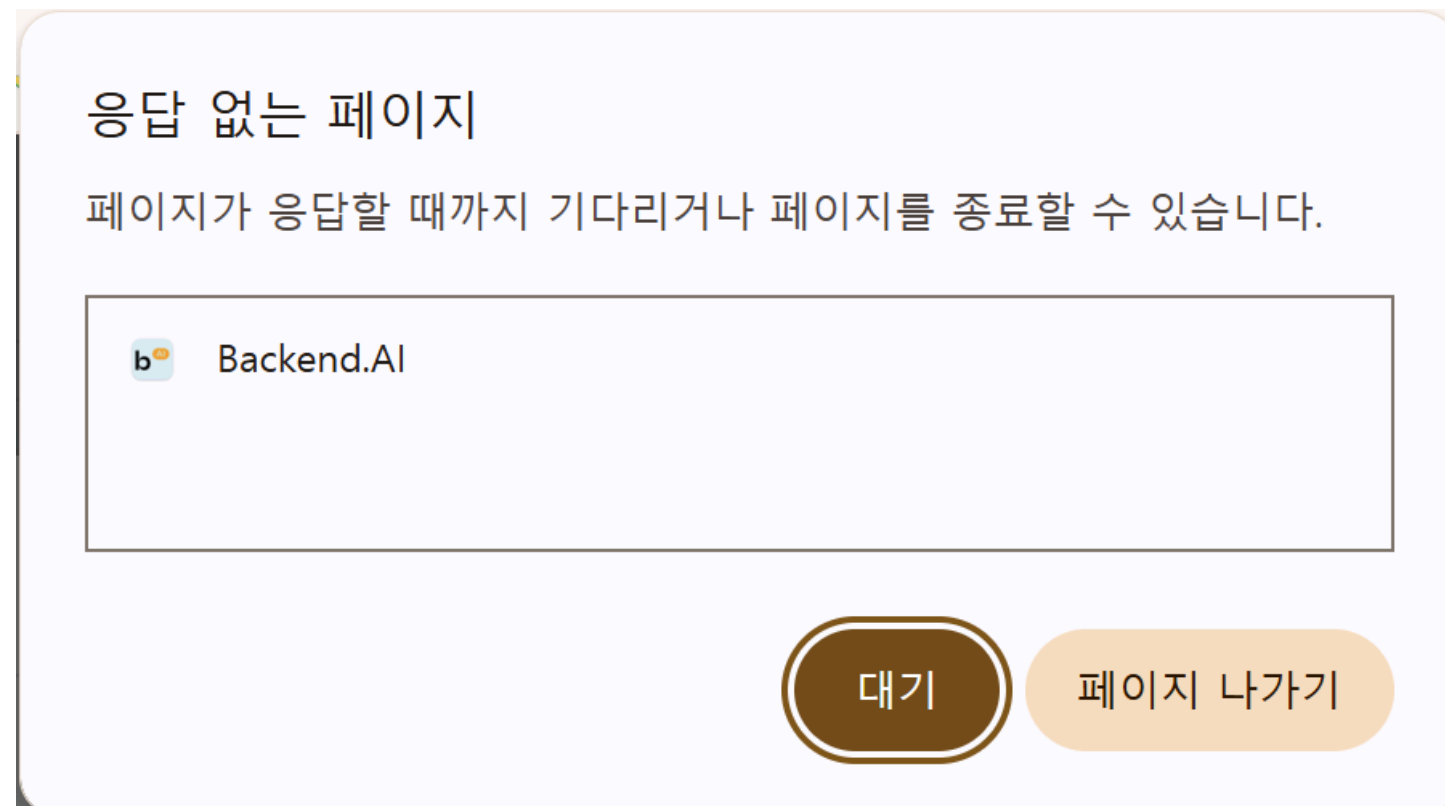
데이터 구성

모든 식재료 데이터는 '1인 가구'라는
목표 대상에 적합하지 않다고 판단하여,
CHAT GPT를 이용해 사용할 식재료를 선택

클래스 목록

```
2 train: train/images
3 val: val/images
4 test: test/images
5 nc: 59
6 names:
7 - sausage
8 - bacon
9 - bean_sprout
10 - beef
11 - bell_pepper
12 - cabbage
13 - carrot
14 - cauliflower
15 - chicken
16 - cucumber
17 - egg
18 - fish
19 - garlic
20 - ginger
21 - kumquat
22 - lemon
23 - onion
24 - pork
25 - potato
26 - shrimp
27 - small_pepper
28 - tofu
29 - tomato
30 - seaweed
31 - PerillaLeaf
32 - eggplant
33 - broccoli
34 - zucchini
35 - anchovy
36 - Mushroom
37 - Pasta
38 - bay_leaf
39 - chickpeas
40 - coriander
41 - long_pepper
42 - mutton
43 - okra
44 - pumpkin
45 - radish
46 - salt
47 - turmeric
48 - pepper
49 - sugar
50 - vinegar
51 - mayonnaise
52 - oil
53 - ketchup
54 - gochujang
55 - lettuce
56 - sesame_oil
57 - soy_sauce
58 - squid
59 - chili_powder
60 - spinach
61 - Chili pepper
62 - Kimchi
63 - Nori
64 - Kelp
65 - Flour
```

서버 이슈 및 로컬 학습 결정



서버 문제 해결 과정

- 학교 GPU 서버에서 업로드 중단·폴더 누락 발생
 - val/test 폴더만 올라가는 오류
 - 용량 초과로 서버 자체가 다운됨
- 결국 전부 로컬 PC에서 직접 학습 진행

모델 선택

- Faster R-CNN / SSD / YOLO 비교

YOLO 계열의 장점

- 1-stage
- 실시간성
- 속도 대비 높은 정확도

YOLOv11 선택시

- 최신 구조
- 우수한 성능

YOLO11n 선택시

- 가장 가벼워 빠르게 학습 가능
- 50 epoch 학습에 약 14시간 소요
- 상위 모델(YOLO11s/m)은 로컬 환경에서 사용하기 어려움

```
# 2) YOLO11n 모델 로드 (pretrained)
model = YOLO("yolo11n.pt")

# 3) 학습
model.train(
    data=DATA_YAML,
    epochs=70,
    imgsz=640,
    batch=16,
    device=0,          # GPU 사용
    project=PROJECT_DIR,
    name=RUN_NAME,
    workers=0,         # 🚫 Windows 멀티프로세싱 문제 피하려면 0 추천
    deterministic=True,
    hsv_h=0.02,
    hsv_s=0.7,
    hsv_v=0.4,
    translate=0.15,
    scale=0.60,
    shear=2.0,
    flipud=0.0,
    fliplr=0.5,
    mosaic=1.0,
    mixup=0.2,
    freeze=10,
    patience=20,
)
```

모델 성능 평가

전체 성능

- mAP50 은 0.793, mAP50-95는 0.607로 전반적인 데이터에 대한 평가 지표는 양호한 편
- mAP@50 = 0.785
- mAP@50-95 = 0.598
- 클래스별 mAP 그래프 또는 표
- 저성능 클래스 4종 mAP < 0.3 표시

Class	Images	Instances	Box(P	R	mAP50	mAP50-95)
all	5912	27765	0.83	0.711	0.793	0.607
sausage	219	547	0.608	0.508	0.634	0.49
bacon	10	18	1	0	0.0467	0.0307
bean_sprout	23	39	0.766	0.335	0.436	0.333
beef	322	484	0.774	0.764	0.813	0.526
bell_pepper	798	946	0.925	0.844	0.909	0.67
cabbage	189	218	0.925	0.958	0.976	0.845
carrot	523	697	0.883	0.738	0.85	0.577
cauliflower	153	165	0.919	0.915	0.979	0.731
chicken	1150	2782	0.937	0.901	0.96	0.745
cucumber	222	431	0.827	0.566	0.733	0.482
egg	168	445	0.86	0.852	0.893	0.66
fish	114	146	0.918	0.822	0.926	0.7
garlic	1285	2053	0.95	0.943	0.969	0.713
ginger	780	1238	0.964	0.982	0.98	0.84
kumquat	218	3793	0.99	0.998	0.992	0.731

저성능 클래스

- mAP50-95가 0.3 밑으로 떨어지는 저성능 클래스만 따로 추출
- 낮은 성능 클래스: bacon, anchovy, long_pepper, squid 등

=== mAP < 0.3 클래스 목록 ===		
01	bacon	0.031
23	seaweed	0.251
28	anchovy	0.000
34	long_pepper	0.160
51	squid	0.214

오버피팅 방지 및 저성능 클래스 개선 전략

저성능 클래스 개선

`Augmentation(hsv_h/s/v, translate, scale, shear, fliplr)`

→ 다양한 색·형태·위치 조건을 학습시켜 작은 클래스의 일반화 능력 강화

`Mosaic-Mixup`

→ 이미지 혼합을 통해 작은 객체, 드문 클래스의 샘플 분포를 확장

오버피팅 방지

`Freeze=10`

→ 모델의 하위 레이어(기본 특징 추출부)를 고정하고 상위 레이어만 미세 조정

→ 불필요한 파라미터 업데이트를 막아 안정적 학습 유도

`Patience=20(Early Stopping)`

→ 성능 개선이 없으면 자동 종료하여 과적합 방지

```
model.train(  
    data=DATA_YAML,  
    epochs=70,  
    imgsz=640,  
    batch=16,  
    device=0,          # GPU 사용  
    project=PROJECT_DIR,  
    name=RUN_NAME,  
    workers=0,         # Windows  
    deterministic=True,  
    hsv_h=0.02,  
    hsv_s=0.7,  
    hsv_v=0.4,  
    translate=0.15,  
    scale=0.60,  
    shear=2.0,  
    flipud=0.0,  
    fliplr=0.5,  
    mosaic=1.0,  
    mixup=0.2,  
    freeze=10,  
    patience=20,  
)
```

OpenAI API 기반 레시피 추천 시스템

OpenAI API

YOLO11n이 감지한 식재료 목록 전달

인식 결과를 OpenAI API(ChatGPT)에 전달

ChatGPT로부터 메뉴 추천 및 레시피 영상 링크를 수신

영상 링크는 Youtube에서 제공하는 API를 사용

FastAPI

FastAPI는 YOLO11n 모델의 인식 결과를 정리해 OpenAI API(ChatGPT)에 전달하여 메뉴를 추천받고, 동시에 추천 메뉴를 기반으로 YouTube API를 호출해 레시피 영상 링크를 조회·반환하는 중간 게이트웨이 역할

```
177 def ask_gpt_for_recipes(ingredients: List[str]) -> List[str]:
178     """
179     GPT에게 재료들 기반으로 '실제로 존재하는 요리 이름' 3개만 추천받기.
180     - 반드시 감지된 재료(ingredients)를 주재료로 쓰는 요리만 추천
181     - 재료 목록에 없는 식재료 이름이 요리 이름에 등장하면 안 됨
182     """
183     if not ingredients:
184         raise ValueError("ingredients 리스트가 비어 있습니다.")
185
186     ingredients_str = ", ".join(ingredients)
187
188     system_prompt = (
189         "너는 1인 가구를 위한 요리 추천 도우미야. "
190         "반드시 '감지된 재료 목록'에 포함된 재료를 주재료로 쓰는 실제 요리만 추천해야 한다. "
191         "예를 들어 감지된 재료가 cabbage(양배추) 하나뿐이라면, 양배추 볶음, 양배추 샐러드, 양배추 겹절이, 양배추 스테이크처럼 "
192         "양배추를 확실히 사용하는 요리만 추천해야 한다. "
193         "감지된 재료에 없는 채소나 재료(예: cucumber/오이, eggplant/가지, spinach/시금치 등)를 요리 이름에 넣으면 안 된다. "
194         "재료 목록에 없는 식재료는 물, 소금, 설탕, 간장, 식용유, 후추 같은 기본 조미료 외에는 사용할 수 없다. "
195         "한국과 일본에서 실제로 자주 먹는 가정식, 분식, 집밥 메뉴만 추천해야 한다. "
196         "재료명을 억지로 붙인 '참기름볶음밥', '간장마요계란밥' 같은 가상의 요리 이름은 절대 추천하면 안 된다. "
197         "반드시 JSON 배열 형식으로만 답해야 한다."
198     )
199
200     user_prompt = f"""
201     YOLO 모델이 아래 재료들을 감지했어 (영어 라벨):
202
203     {ingredients_str}
204
205     규칙은 다음과 같아:
206
207     1) 반드시 위 재료들 중 하나 이상을 '주재료'로 사용하는 실제 요리만 추천할 것.
208     2) 재료 목록에 없는 채소나 재료(예: 오이, 가지, 시금치, 파프리카 등)는 요리 이름에 포함되면 안 된다.
209     3) 물, 소금, 설탕, 후추, 간장, 고추장, 식용유 같은 기본 양념은 추가로 사용해도 되지만, 요리 이름에 넣지 마.
210     4) 한국 또는 일본에서 실제로 존재하고 많이 먹는 요리만 추천할 것.
211     5) 레시피 설명, 과정, 문장, 설명 문구는 쓰지 말고, 오직 요리 이름만 배열로 보낼 것.
212
213     예시:
214     - 재료: cabbage → 추천 예: ["양배추볶음", "양배추샐러드", "양배추겹절이"]
215     - 재료: egg, rice, kimchi → 추천 예: ["김치볶음밥", "계란볶음밥", "김치전"]
216
217     반드시 아래 형식으로만 대답해:
218
219     ["요리1", "요리2", "요리3"]
220     """
```

GPT 레시피 추천 예시

GPT 레시피 추천 결과

- 인식한 재료: [예: cabbage, egg, pork]
- 추천된 레시피 예 :
 - 양배추 볶음
 - 계란찜
 - 돼지고기덮밥

결론 - 한계점

1. 선별할 데이터 기준 불명확
2. 적절한 데이터 부족
3. 재료의 정량적 파악 어려움

개선방안

- ✓ 시간이 더 주어진다면 더 넓은 범위의 데이터를 사용하여, 인식되지 않는 식재료의 종류를 줄인다.
- ✓ 자취생이 주로 소비하는 라면, 참치캔과 스팸 등의 가공 식품들을 추가한다.
- ✓ 모델이 대략적인 양을 추정하고, 사용자가 마지막으로 한 번 조정해 줌으로써 정량 정보의 불확실성을 보완한다.



Q&A Session